SMICES | 2026–2027

AI for Cybersecurity

C4 | Convolutional Neural Networks

Ana Nicolae, Radu Bolborici, Vlad Vasilescu
11 November 2026

National University of Science and Technology POLITEHNICA Bucharest
Faculty of Electronics, Telecommunications and Information Technology

Overview



1 Convolution Operator

2 Convolutions in Deep Neural Networks

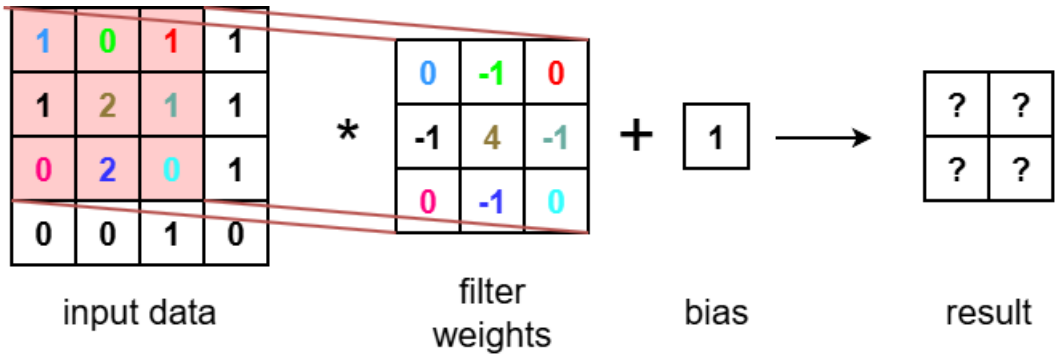
3 Parameter Efficiency

4 10 years of CNN revolution

5 Practical Insights for Model Training and Debugging

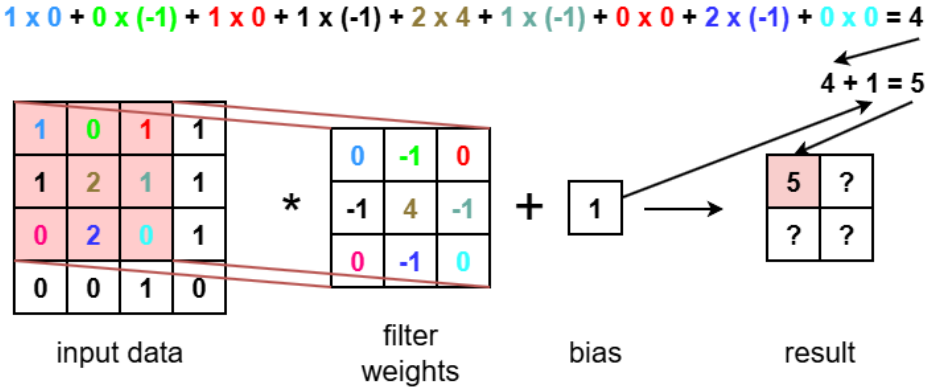
2D Convolution

- How to apply 2D convolution? Numerical Example.



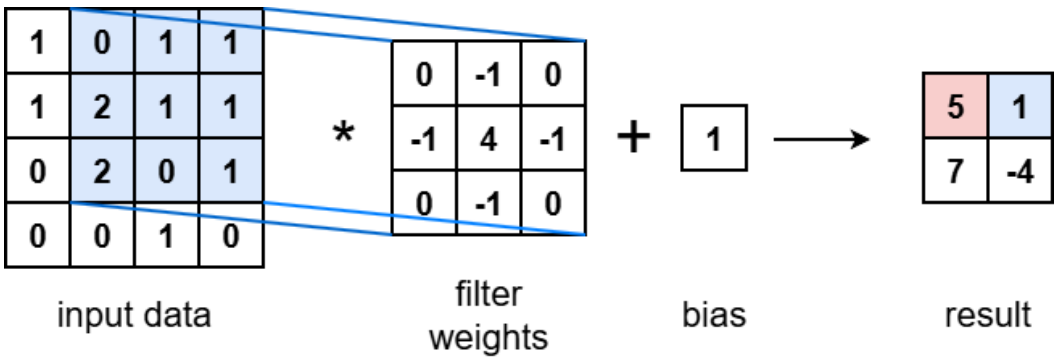
2D Convolution

- **Element-wise multiplication** followed by **summation** is performed between the filter weights and the overlapping input region. Finally, the bias is added.



2D Convolution

- The filter **slides** along the image, from left to right and then top-down.



2D Convolution - Visualizing

Visualizing convolution. Terminology.

- The **kernel size** k is the dimension of the filter (eg., $2 \times 2, 3 \times 3, 5 \times 5$).
- The **stride** is the number of positions the filter **slides** for one step.
- The **padding** specifies how many pixels (usually zeros) are added around the input borders, controlling the output size. Using padding, *the output can be kept the same size as the input*.
- Check **Ezyang's Conv-2D Visualizer** to see how different-sized kernels slide along different sized inputs and understand **kernel size**, **stride** and **padding**.

- Check this guide for info about how to compute output size.

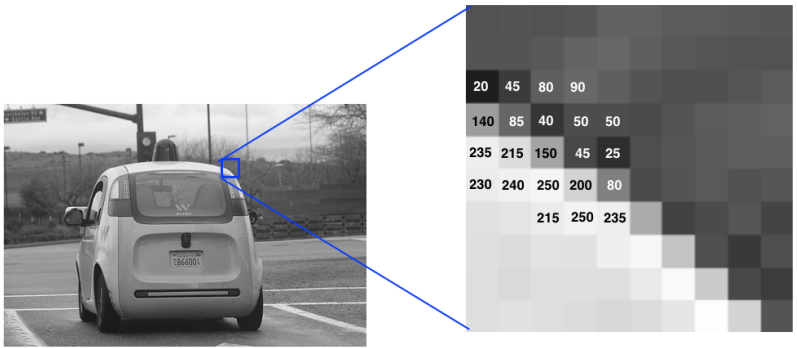
2D Convolution - Input Data

QUESTION: What could the 2D input data matrix represent?

2D Convolution - Input Data Grayscale Image

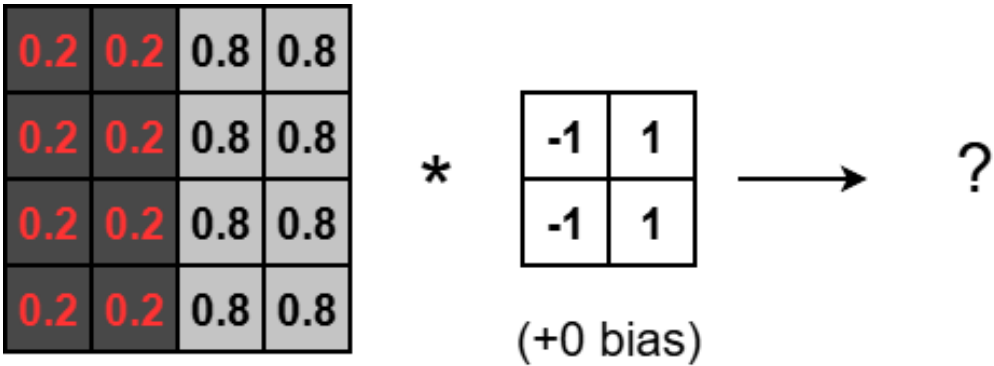
QUESTION: What could the 2D input data matrix represent?

- **EXAMPLE**: Each matrix element can be the gray level in a **greyscale image**.



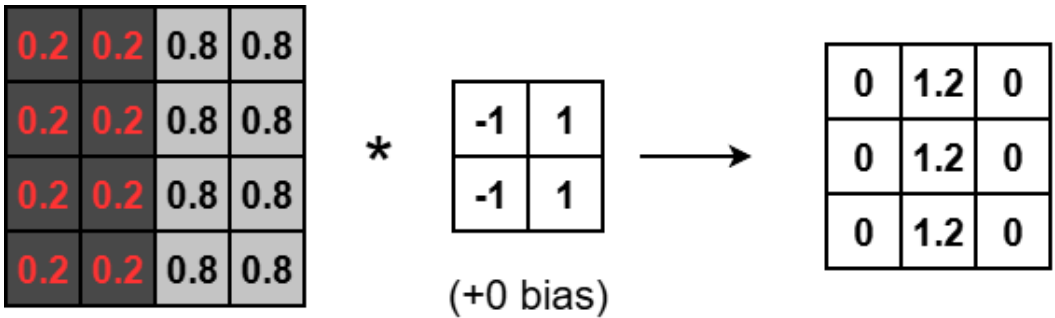
2D Convolution - Numerical Exercise

- Given the following input and 2×2 filter (with bias 0), compute the convolution result. Stride is set to 1 and there is no padding.



2D Convolution - Numerical Exercise

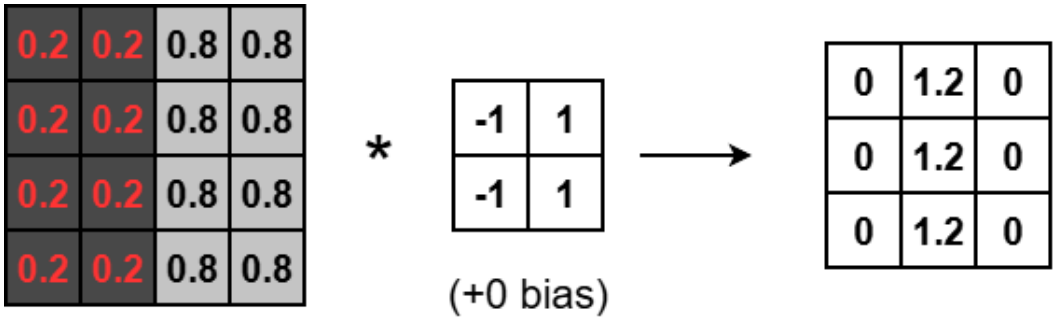
QUESTION: What does the convolution result tell us about the image?



2D Convolution - The Filter as a Feature Detector

QUESTION: What does the convolution result tell us about the image?

- **ANSWER**: For this particular filter, the output contains high values in the places where vertical transitions are located. The given filter **detects vertical contours**.



2D Convolution - Examples of Handcrafted Filters

- **CORE IDEA**: The filter controls what features are detected.
- We can build filters that detect **vertical**, **horizontal** or **diagonal contours**.

-1	0	1
-1	0	1
-1	0	1

Vertical contour detector

-1	-1	-1
0	0	0
1	1	1

Horizontal contour detector

-1	-1	0
-1	0	1
0	1	1

Diagonal contour detector /

0	1	1
-1	0	1
-1	-1	0

Diagonal contour detector \

Convolution Result on a Real Image



Sobel filters applied to a 512×512 image: one for vertical edges (G_x), one for horizontal (G_y) (The image on the right is summation of the two convolution results).

2D Convolution - Learned Filters

- However, in **convolutional neural networks**, the filters are **not handcrafted**. Instead, they are **learned automatically**.
- The **filter weights and biases** are optimized during training using backpropagation and optimization algorithms (e.g., SGD, Adam), just like the parameters of fully connected layers.

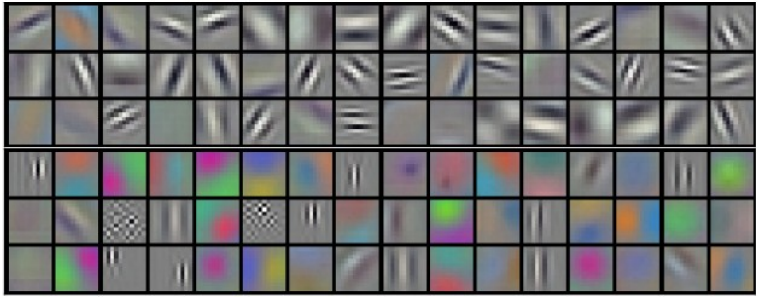
$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

$$\boxed{b}$$

Example, for GD

$$\begin{cases} w_i = w_i - \eta \dfrac{\partial L}{\partial w_i} \\ b = b - \eta \dfrac{\partial L}{\partial b} \end{cases}$$

Supervision



Heatmaps showing what filters from the first layer look like.

2D Convolution - Mathematical Formulation

- **2D convolution:**

$$S(i,j) = (K * I)(i,j) = \sum_m \sum_n I(i-m, j-n) K(m,n)$$

- **Cross-correlation (as actually used in CNNs):**

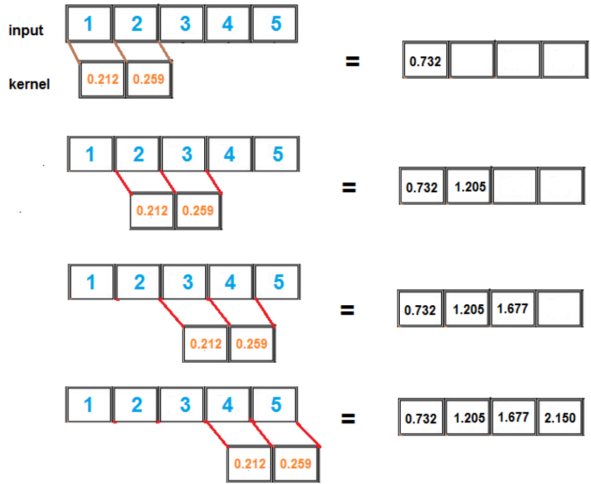
$$S(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n) + b$$

where I is the input tensor, K is the filter and $S(i,j)$ is the output, for position (i,j). The bias b is used in practice, but not included in the theoretical formulation of the convolution operator.

- In practice, CNNs compute cross-correlation — not true mathematical convolution
- True mathematical convolution would require rotating the filter $180\,\mathrm{deg}$

1D Convolution

- Convolution can also be applied to **1D signals** using **1D kernels**.
- Commonly used for **audio**, **time series**, and **sensor data**.



Convolution as Matrix Multiplication

Convolution can be expressed as matrix multiplication using a structured matrix called a Toeplitz matrix.

- The sliding window operation can be reformulated as a matrix product.
- This is mostly used for mathematical analysis, not for efficient computation.
- The kernel is transformed into a **Toeplitz-like matrix**, which encodes all shifted versions of the kernel.
- As input size grows and kernel size remains fixed, the matrix becomes **very sparse**.

- Input: $[x_1, x_2, x_3, x_4]$
- Kernel: $[k_1, k_2, k_3]$

$$\begin{bmatrix} k_1 & k_2 & k_3 & 0 \\ 0 & k_1 & k_2 & k_3 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

- The **Toeplitz-like matrix** shows that convolution can be seen as a **fully connected layer** with:
  - a lot of weights set to 0
  - a lot of weights shared across positions

Convolution for Coloured Images

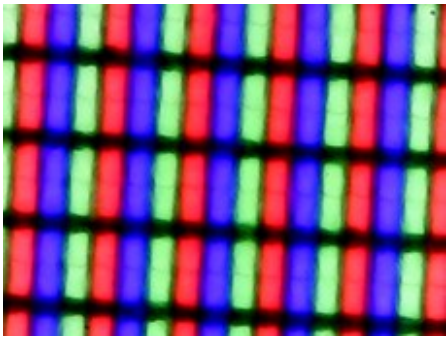
- Greyscale images can be represented using a 2D matrix.

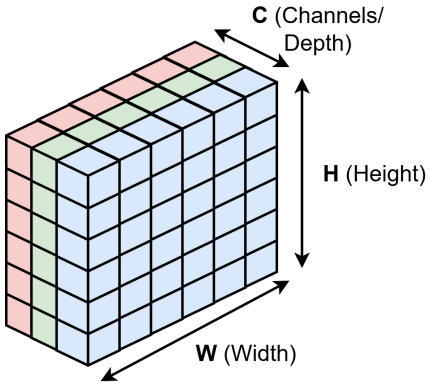
  QUESTION: How are coloured images represented?

Convolution for Coloured Images

QUESTION: How are coloured images represented?

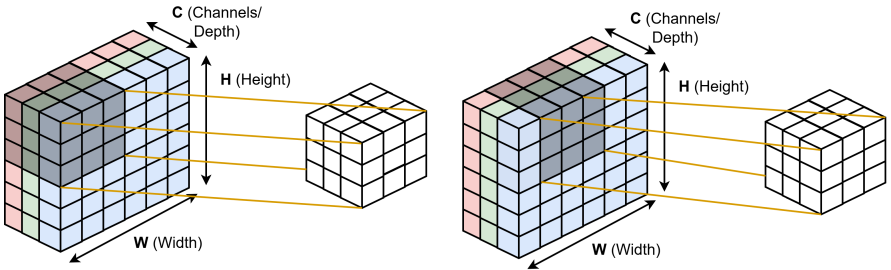
- **ANSWER**: Each pixel (spatial location) has three associated values, for Red (R), Green (G) and Blue (B) intensity levels.





Convolution for Coloured Images

- The **kernel depth** matches the **number of input channels** (e.g., 3 for RGB).
- The kernel slides only along the **spatial dimensions** (H, W), not along the channel dimension (for standard convolution).



The Convolutional Layer

- Each filter detects **one pattern** (e.g., vertical contours).

QUESTION: Is one single filter enough to classify complex objects?

The Convolutional Layer

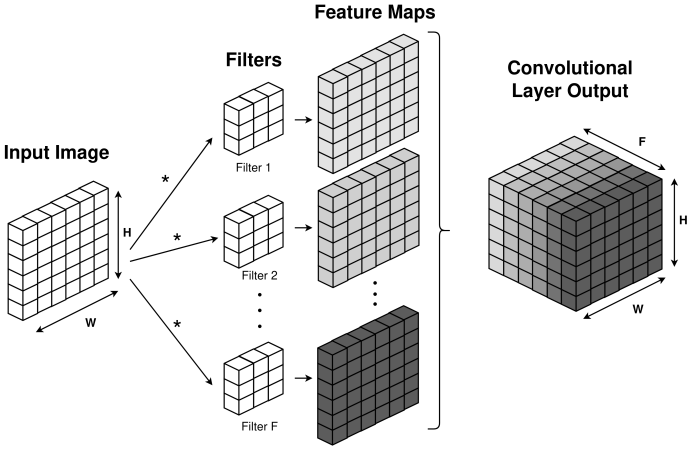
- Each filter detects **one pattern** (e.g., vertical contours).

> **QUESTION:** Is one single filter enough to classify complex objects?

- For building a ***convolutional layer*, multiple filters are used** – each filter *learns* to detect a specific pattern. The number of filters F on a convolutional layer is a hyperparameter.
- The result of convolution between the input and a filter is called a **feature map**, because it shows where a specific feature is located.
- The feature maps obtained by a layer's filters are **stacked along the channel dimension**.
- *Note: This is a simplification. In practice, a filter may respond to multiple features, and different filters may learn to detect similar patterns.*

The Convolutional Layer

- For building a ***convolutional layer*, multiple filters are used** – each filter *learns* to detect a specific pattern. The number of filters F on a convolutional layer is a hyperparameter.
- The result of convolution between the input and a filter is called a **feature map**, because it shows where a specific feature is located.
- The feature maps obtained by a layer's filters are **stacked along the channel dimension**.



The Convolutional Layer

QUESTION: Is a convolution layer detecting various simple shapes (vertical, horizontal, diagonal contours) sufficient to classify complex objects?

The Convolutional Layer

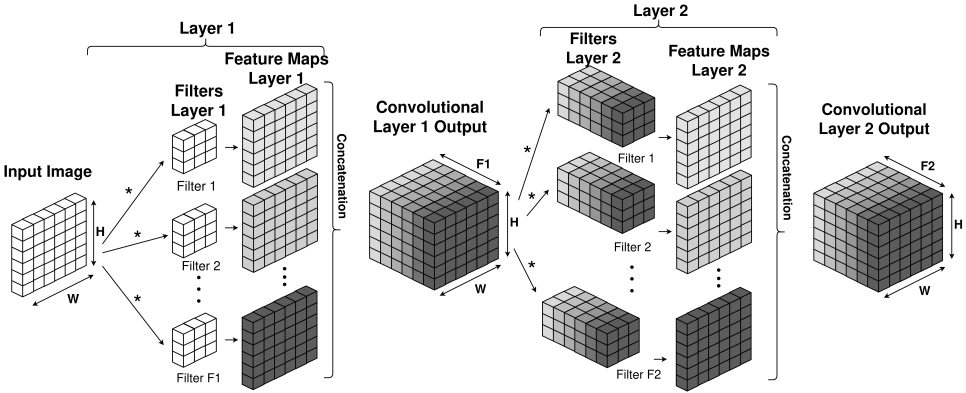
QUESTION: Is a convolution layer detecting various simple shapes (vertical, horizontal, diagonal contours) sufficient to classify complex objects?

- Maybe. It depends on the problem.

- CNNs typically contain **multiple convolutional layers**. Each layer takes the **feature maps produced by the previous layer** as input and learns to detect **more complex patterns** by combining information from those feature maps.
- Each filter must span **all input channels**. Therefore, the **depth of each filter** must equal the **number of filters in the previous layer** (i.e., the number of input feature maps) – just like in the RGB input example.

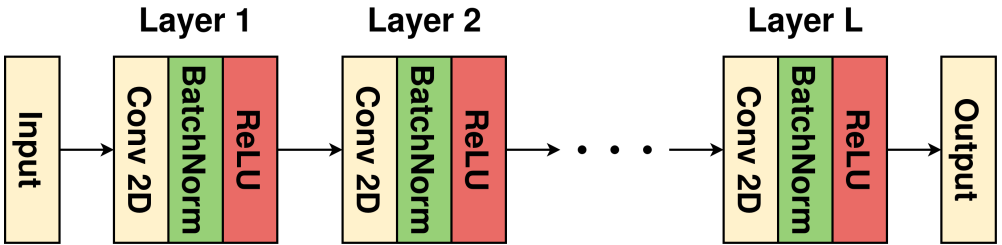
Multi-Layer CNN

- Each layer takes the **feature maps produced by the previous layer** as input.
- Therefore, the **depth of each filter** must equal the **number of filters in the previous layer** (i.e., the number of input feature maps).



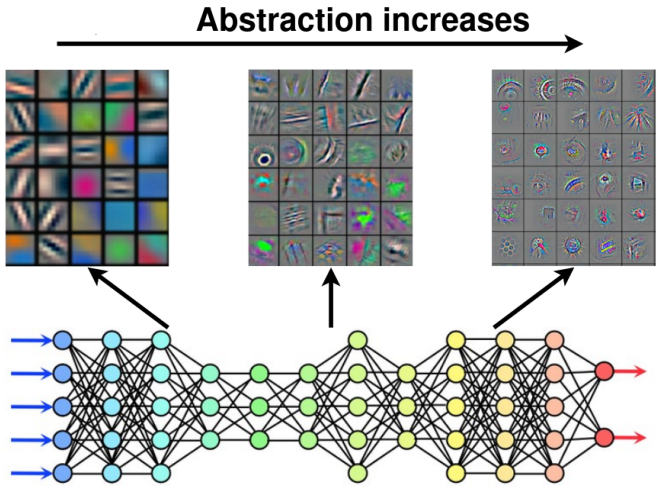
Multi-Layer CNN – Activation Functions

- Since **convolution is a linear operation**, **non-linear activation functions** (e.g., ReLU) are needed to learn complex, non-linear relationships.
- **Batch Normalization** is also often used to **stabilize and accelerate training** by normalizing the activations within a mini-batch.
- Each layer may have its own number of filters. The **number of layers** and the **number of filters per layer** are **hyperparameters** to tune.



Multi-Layer CNN – Increasing Depth

- Layers closer to the input learn to detect **low-level features**, such as edges, contours, corners, and simple textures.
- Intermediate layers **combine low-level features** to detect more complex patterns, such as shapes, textures, and object parts.
- Deeper layers learn **high-level, semantic features**, representing more abstract concepts, such as objects or distinctive object characteristics.



Multi-Layer CNN - Memorization vs Abstraction

- Increasing the **number of filters** of a layer increases the **number of patterns** the layer can detect.

> **QUESTION:** Is a larger number of filters always better?

Multi-Layer CNN - Memorization vs Abstraction

- Increasing the **number of filters** of a layer increases the **number of patterns** the layer can detect.

  QUESTION: Is a larger number of filters always better?

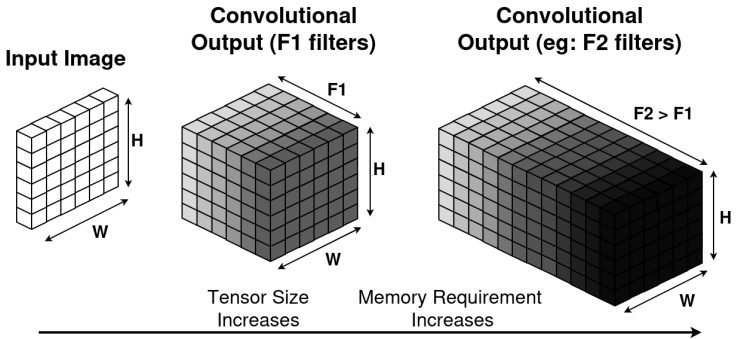
  - If the number of filters is too high, the model may **overfit** to the training data, because the model may learn features that are very specific to the examples in the training set, rather than general features. Reducing the number of filters may force the model to only learn the most relevant features that generalize in a wide range of situations.
  - However, too few filters may lead to **underfitting**: the model may lack the capacity to learn all the relevant patterns.
  - The optimal number of filters depends on the problem.

Multi-Layer CNN – Increasing Filters

- Building CNNs with the **same number of filters** in every layer is possible and can work well.
- However, a common architectural choice is to **increase the number of filters in deeper layers**, allowing them to learn a richer variety of complex patterns.

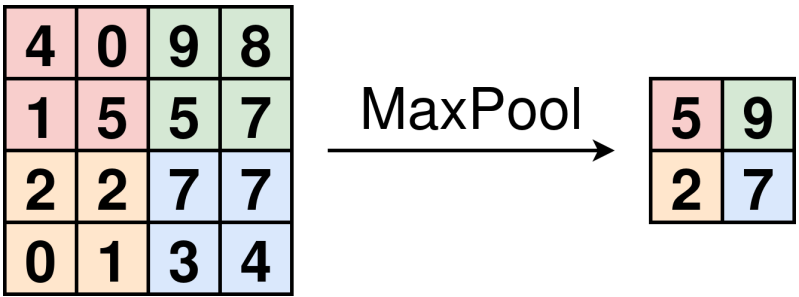
Multi-Layer CNN – Increasing Filters

- Building CNNs with the **same number of filters** in every layer is possible and can work well.
- However, a common architectural choice is to **increase the number of filters in deeper layers**, allowing them to learn a richer variety of complex patterns.
- **PROBLEM**: Increasing the number of filters **increases the data tensor size and the memory requirement**. This is because the feature maps are concatenated.



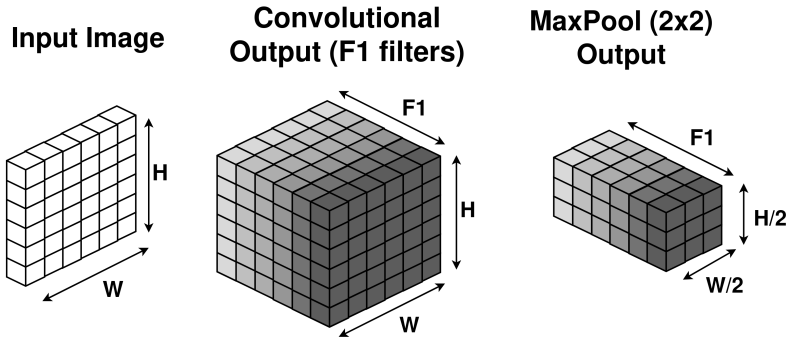
Multi-Layer CNN – Pooling

- **PROBLEM**: Increasing the number of filters **increases the data tensor size and the memory requirement**.
- **SOLUTION**: Reducing the spatial resolution (reducing W and H) for specific layers. Such architectures decrease spatial dimension, while increasing channel depth.
- One common operator is **MaxPooling (MaxPool)**, which divides each feature map into local regions and **retains only the maximum value** from each region, reducing its spatial dimensions.



Multi-Layer CNN – Pooling

- One common operator is **MaxPooling (MaxPool)**, which divides each feature map into local regions and **retains only the maximum value** from each region, reducing its spatial dimensions.



Multi-Layer CNN - Increasing Network Depth

- Increasing the **number of layers** allows the CNN to learn **increasingly complex patterns** in deeper layers.

> **QUESTION:** Is a larger number of layers always better?

Multi-Layer CNN - Increasing Network Depth

- Increasing the **number of layers** allows the CNN to learn **increasingly complex patterns** in deeper layers.

  QUESTION: Is a larger number of layers always better?

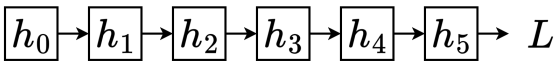
  - Regardless of the layer type (convolutional or fully connected), gradients are propagated backward through the network using the **chain rule**.

    QUESTION: As we move toward the input layers, **what happens to the magnitude of the gradients**, especially in very deep networks?

Multi-Layer CNN - Increasing Network Depth

QUESTION: As we move toward the input layers, **what happens to the magnitude of the gradients**, especially in very deep networks?

- When the derivatives involved in backpropagation have **small absolute values** (typically below 1), their repeated multiplication can lead to **increasingly smaller gradients** as we move toward the input layers. This phenomenon is called the **vanishing gradient** problem.



$$\frac{\partial L}{\partial h_5} \to 0.1 \quad \text{(supposed values)}$$

$$\frac{\partial L}{\partial h_4} = \underbrace{\frac{\partial L}{\partial h_5}}_{0.1} \cdot \underbrace{\frac{\partial h_5}{\partial h_4}}_{0.1} \approx 0.1^2$$

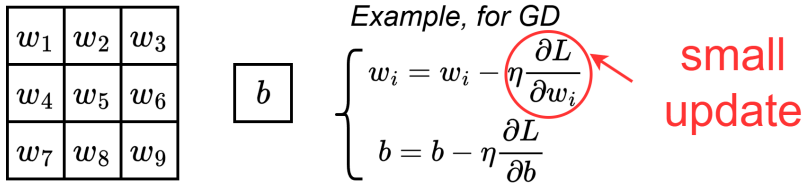
$$\vdots$$

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_5} \cdot \frac{\partial h_5}{\partial h_4} \cdot \frac{\partial h_4}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial h_0} \approx 0.1^6$$

Multi-Layer CNN - Increasing Network Depth

QUESTION: As we move toward the input layers, **what happens to the magnitude of the gradients**, especially in very deep networks?

- When the derivatives involved in backpropagation have **small absolute values** (typically below 1), their repeated multiplication can lead to **increasingly smaller gradients** as we move toward the input layers. This phenomenon is called the **vanishing gradient** problem.
- **PROBLEM**: if the gradients of the parameters in the input layers are very small, their **updates will also be very small**, which makes **learning very slow**.



Multi-Layer CNN - Residual Connections

- **PROBLEM**: if the gradients of the parameters in the input layers are very small, their **updates will also be very slow**, which limits learning.
- **SOLUTION**: **Residual connections**, introduced by He et. al. in 2016, the winner of ILSVRC'16.
- The introduced network is called **ResNet**. It allows the **input of a block of layers to be added directly to its output**, creating a shortcut that facilitates gradient propagation during backpropagation.

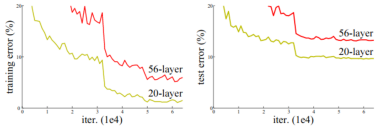
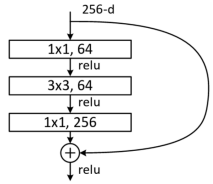


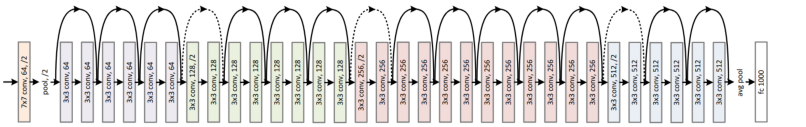
Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Deeper is worse ?!



Residual block

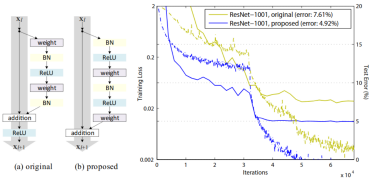
Residual Networks



ResNet34. Dotted shortcuts and conv"/2" are stride 2 to match the spatial dimensions. Dotted shortcuts use 1×1 conv to match the depth. 0.46M parameters.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2.x	56×56	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$
conv3.x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 8$
conv4.x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 36$
conv5.x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Resnet architectures. Conv are "Conv-BN-Relu". ResNet-50 has 23M parameters.



Shortcut variations (He et al., 2016b)

Other Variants of Skip Connections

Highway Networks (Srivastava et al., 2015)

Use **learnable gates** (similar to LSTMs) to control how much information is **transformed** versus **preserved**.

- **Transform gate** $T(x)$: controls how much of the **transformed input** $H(x)$ is used.
- **Carry gate** $C(x) = 1 - T(x)$: controls how much of the **original input** x is preserved.

$$y = T(x) \odot H(x) + C(x) \odot x$$

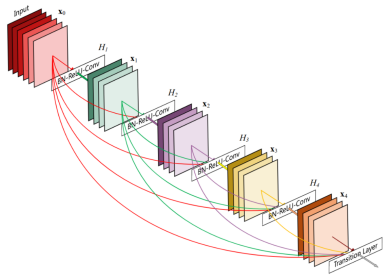
- $T(x) \approx 0$: mostly **preserve** x.
- $T(x) \approx 1$: mostly **transform** x.

DenseNets (Huang et al., 2017)

Each layer receives the **feature maps produced by all preceding layers**.

- Feature maps are **concatenated along the channel axis**.
- Encourages **feature reuse** and improves **gradient propagation**.

$$x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$$

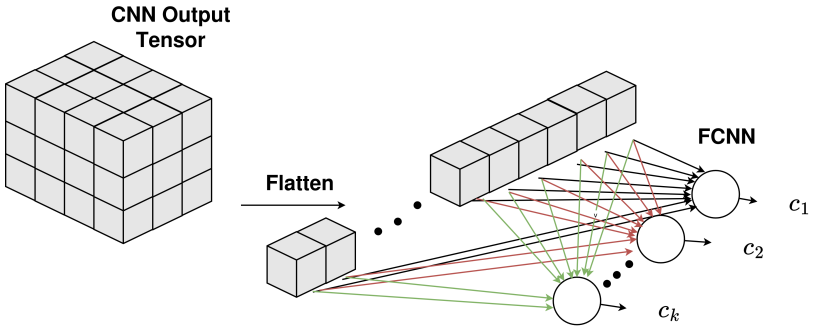


Multi-Layer CNN - classification

- **PROBLEM**: Suppose we have built a CNN architecture and want to use it for **image classification** (k classes). How can we transform its output feature maps into **class predictions**?

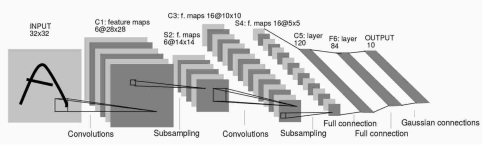
Multi-Layer CNN - classification

- **PROBLEM**: Suppose we have built a CNN architecture and want to use it for **image classification** (k classes). How can we transform its output feature maps into **class predictions**?
- **SOLUTION**: Flatten the CNN output tensor and send the data to a fully connected neural network. This is used by the architecture of LeNet-5, proposed by LeCun in 1989.



LeNet5 architecture

- The architecture of LeNet-5, proposed by LeCun in 1989.



LeNet5 (LeCun et al., 1989)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

TABLE I

EACH COLUMN INDICATES WHICH FEATURE MAP IN S2 ARE COMBINED BY THE UNITS IN A PARTICULAR FEATURE MAP OF C3.

Architecture:

Two main parts:

- Convolutional part
- Fully connected part

Specificities:

- Weighted sub-sampling
- Gaussian connections (RBF output layer)
- connectivity pattern ($S_2 - C_3$) to reduce the number of weights

Number of parameters:

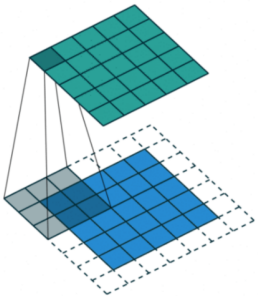
C1	156
S2	12
C3	1.516
S4	32
C5	48.120
F6	10.164

Receptive field

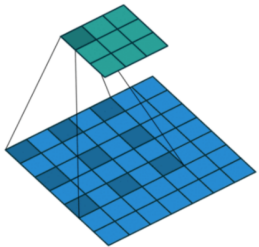
- The ***receptive field*** *refers to the area of the input image that a particular neuron in the network "sees"*.
- It represents the region in the input space that contributes to the activation of a given neuron.
- Receptive fields can be local or global, depending on the architecture and layers of the neural network.
- Understanding the receptive field is crucial for interpreting the behavior and features learned by different layers in a convolutional neural network.
- It plays a key role in tasks like object detection, where the network must recognize objects at different scales and positions within an image.
- For calculating the effective receptive field size, see for example this calculator.

A-TROU Convolutions

- The effective receptive field can grow faster with a-trou convolutions (or dilated convolutions).
- Used by Yu and Koltun in 2016.
- Check **Ezyang's Conv-2D Visualizer** to see how dilated convolution works.



Conv 3, Pad 1, Stride 1



Conv 3, Pad 0, Stride 1, Dilated 1

After all: Why Convolutions?

QUESTION: After all, why choose to use **convolution layers** instead of **fully connected layers**?

After all: Why Convolutions?

QUESTION: After all, why choose to use **convolution layers** instead of **fully connected layers**?

- Think about a fully connected layer with N neurons operating on a greyscale 1000×1000 image. What is its parameter count?

After all: Why Convolutions?

QUESTION: After all, why choose to use **convolution layers** instead of **fully connected layers**?

- Think about a fully connected layer with N neurons operating on a greyscale 1000 image. What is its parameter count?
- 1000 input neurons $\rightarrow (1,000 \times 1,000 + 1)N = 1,000,001N$ parameters (including the bias)
- Now think about a convolutional layer with 32 filters sized 3×3. How many parameters does it have?

After all: Why Convolutions?

QUESTION: After all, why choose to use **convolution layers** instead of **fully connected layers**?

- Think about a fully connected layer with N neurons operating on a greyscale 1000 image. What is its parameter count?
- 1000 input neurons $\rightarrow (1,000 \times 1,000 + 1)N = 1,000,001N$ parameters (including the bias)
- Now think about a convolutional layer with 32 filters sized 3×3. How many parameters does it have?
- Each filter has $3 \times 3 = 9$ weights and one bias, therefore the total is $10 \times 32 = 320$

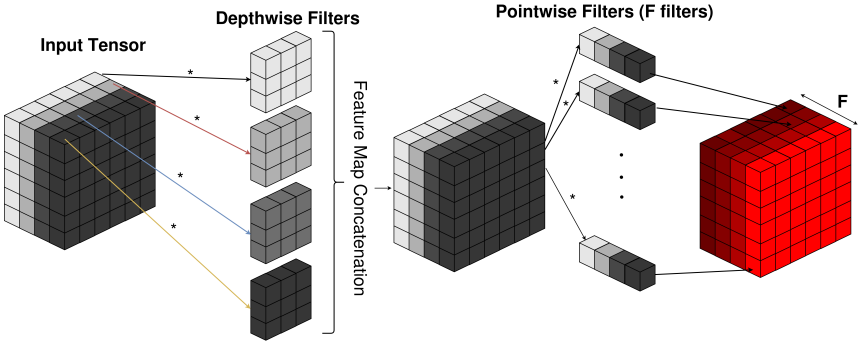
CONCLUSION: CNNs are much more parameter efficient compared to fully connected layers.

Convolution Advantages

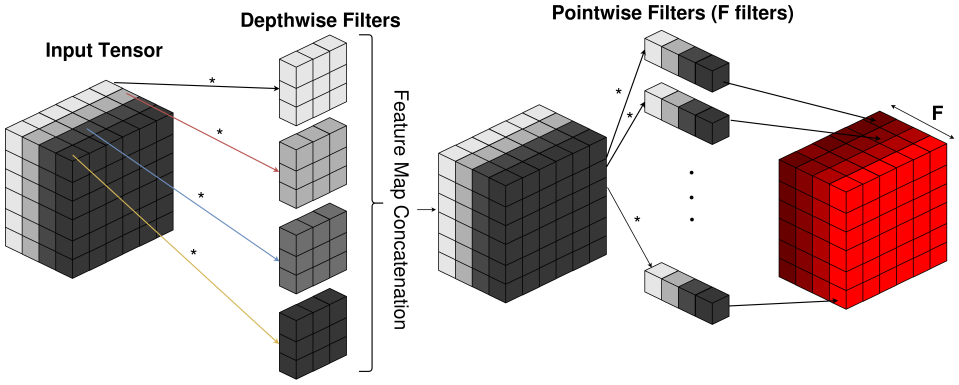
- **1. Parameter Efficiency**: CNNs are much more parameter efficient compared to fully connected layers.
- **2. Translation Equivariance**: the same feature can be detected regardless of its position in the image.
- **3. Exploiting Spatial Structure (Local Connectivity)**: nearby pixels are processed together, preserving spatial relationships.

Depthwise Separable Convolutions

- **Depthwise convolution**: each filter is 2D and is applied exclusively to its assigned input channel. Information from different channels doesn't mix. Their feature maps are stacked.
- **Pointwise convolution**: Each 1×1 filter spans **all input channels**, combining their information to produce **one output feature map**. The resulting feature maps are **stacked along the channel axis**.



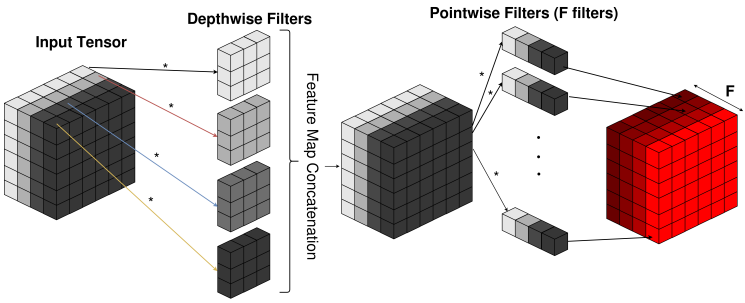
Depthwise Separable Convolutions



QUESTION: How many parameters does one **depthwise separable** layer have?

- Consider: K = Depthwise filter size; C_{in} = input channels; C_{out} = output channels.

Depthwise Separable Convolutions

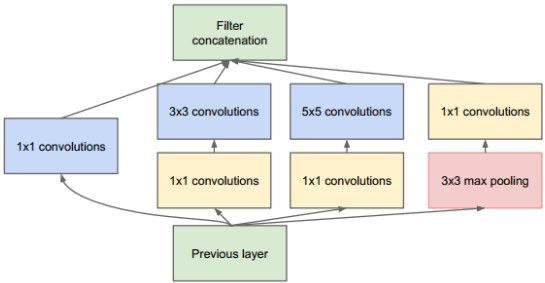


QUESTION: How many parameters does one **depthwise separable** layer have?

- Consider: $K =$ Depthwise filter size; $C_{in} =$ input channels; $C_{out} = F =$ output channels.
- Depthwise layer: $K^2 C_{in}$
- Pointwise layer: $C_{in} C_{out}$
- Depthwise Separable layer: $K^2 C_{in} + C_{in} C_{out}$
- Normal convolutional layer: $K^2 C_{in} C_{out}$ (without considering biases)
- Since C_{in} and C_{out} are typically large: $K^2 C_{in} C_{out} >> K^2 C_{in} + C_{in} C_{out}$

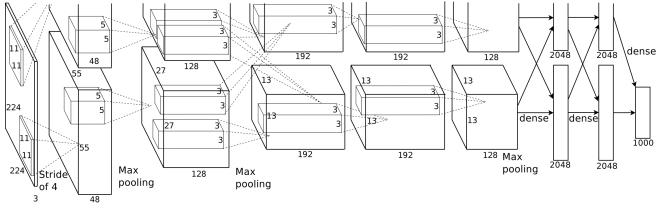
Multi-Scale Feature extraction

- **Main idea**: The **Inception module**, introduced by Szegedy et al. (2014), extracts features at **multiple spatial scales** by applying different operations in parallel.
- **Channel reduction**: 1×1 convolutions are applied before expensive convolutions to **reduce the number of input channels**, decreasing the number of parameters and computational cost.
- **Multi-scale feature extraction**: Parallel branches use different kernel sizes (1×1, 3×3, 5×5) and MaxPooling to capture patterns from **different spatial regions**.
- **Output**: The resulting feature maps are **concatenated along the channel axis**.



AlexNet

Introduced by Krizhevsky et. al. in 2012, considered the "spark" giving birth to the revival of neural networks.



- Top 5 error of 16%, runner-up at 26%
- Several convolutions stacked before pooling
- Trained on 2 GPUs, for a week on ImageNet (resized to $256 \times 256 \times 3$), 1M images. (now it's 18 minutes).
- 60 Million parameters, dropout, momentum, L2 penalty, dataset augmentation (rand crop (224×224) , translation, reflections, PCA
- Learning rate 0.001 at divided by 10 when validation error stalls
- At test time, avg probabilities on crops + reflections

VGG Net

Introduced by Simonyan and Zisserman in 2015, the winner of ILSVRC'14.

- 16 layers : 13 convolutional, 3 FC
- 3×3 convolutions, 2×2 pooling
- Stacked 3×3 convolutions $\equiv 5 \times 5$ convolution receptive field
  - If $c_{in} = K$, $c_{out} = K$, 5×5 conv. $\rightarrow 25K^2$ parameters
  - If $c_{in} = K$, $c_{out} = K$, 2 stacked 3×3 conv. $\rightarrow 18K^2$ param.
- 140 million parameters, batch size(256), Momentum(0.9), Weight decay (0.0005), Dropout(0.5) in FC, learning rate (0.01) divided 3 times by 10.
- Initialization of B, C, D, E from trained A. Init. A random from $\mathcal{N}(0, 10^{-2}, b = 0)$
- can cope with variable input size changing the FC layers to conv 7×7, conv 1×1.

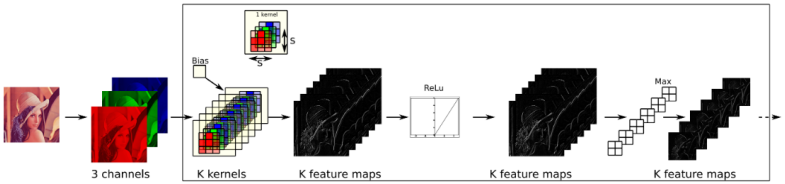
Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv(receptive field size)-(number of channels)". The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

CNN Definitions



The building blocks of the convolutional part of a vanilla CNN

Convolution:

- size (e.g. 3×3, 5×5)
- padding (e.g. $1, 2$)
- stride (e.g. 1)

Pooling (max/average):

- size (e.g. 2×2)
- padding (e.g. 0)
- stride (e.g. 2)

We work with 4D tensors for 2D images, 3D tensors for nD temporal series (e.g. multiple simultaneous recordings), 2D tensors for 1D temporal series.

In Pytorch, the tensors follow the Batch-Channel-Height-Width (**BCHW**, channel-first) convention. Other frameworks, like TensorFlow or CNTK, use **BHWC** (channel-last)

Using pre-trained models

- **PROBLEM**: We have **limited training data**.
- **SOLUTION 1**: **Transfer learning**. All modern frameworks provide a **model zoo** of pre-trained networks. See for example torchvision documentation whre you can find pretrained models for classification, detection, segmentation ...
- What is common practice is to cut the head and **finetune the only the output layer**, while the rest of the layers stay **frozen**.
- Do not forget about input normalization!

```
import torchvision.models as models

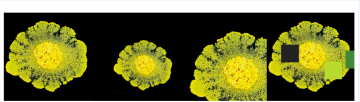
resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
mobilenet = models.mobilenet_v2(pretrained=True)
```

Dataset Augmentation

- **SOLUTION 2**: **Augmentation**: apply transforms to existing data, in order to reduce overfit to aprticular features (e.g. color jittering, translations, reflections, rotations, PCA, ...) on the inputs that make predictable changes on the targets.

Examples

- Use imgaug library



- The images represent all *physarum polycephalum*, and was taken from CNRS
- Another interesting work around this subject is mixup.
- Make sure that the augmentation transforms make sense.



Example on CIFAR-100

- Has 100 object classes, each having 600 images per class.
- Size: $32 \times 32 \times 3$
- The training set has 500×100 images, and the test set has 100×100 images



Model Architecture – An Example

Operator	Res.	RF Size	#Ch
ConvBlock	32×32	5×5	32
ConvBlock	32×32	9×9	32
Sub	16×16	15×15	32
ConvBlock	16×16	15×15	128
ConvBlock	16×16	23×23	128
Sub	8×8	31×31	128
AvgPool	1×1		128
Linear	100		

ConvBlock: 2x
[Conv(1x3)-(BN)-Relu-Conv(3x1)-(BN)-Relu]
Sub : Conv(3x3, stride=2)-(BN)-Relu
No. of. param: $\approx 2M$

Settings:

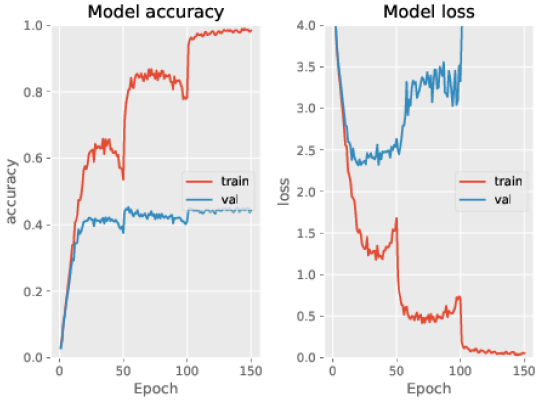
- BatchSize(32)
- SGD(lrate=0.01) with momentum(0.9)
- learning rate halved every 50 epochs
- validation on 20%, early stopping on the val loss

Other configurations:

- base
- Conv-BN-Relu or Conv-Relu
- dataset augmentation (HFlip, Trans(5pix), Scale(0.8,1.2)), CenterCrop(32)
- Dropout, L2, label smoothing

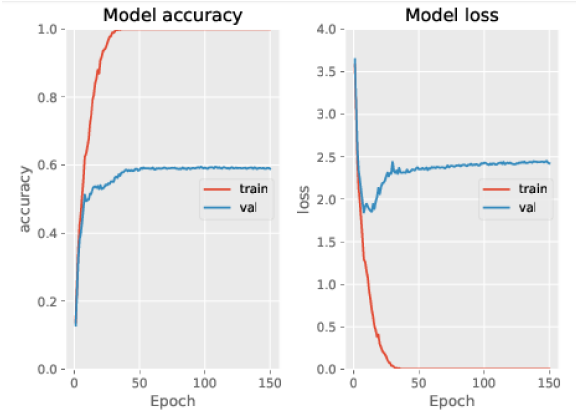
Baseline

- No regularization, no BatchNorm,
- Observe that **training convergence is not very smooth**.



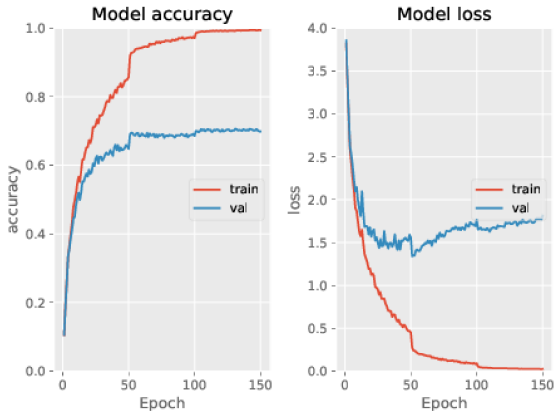
with BatchNorm

- with BatchNorm after each convolution,
- Observe that **convergence became smoother**.



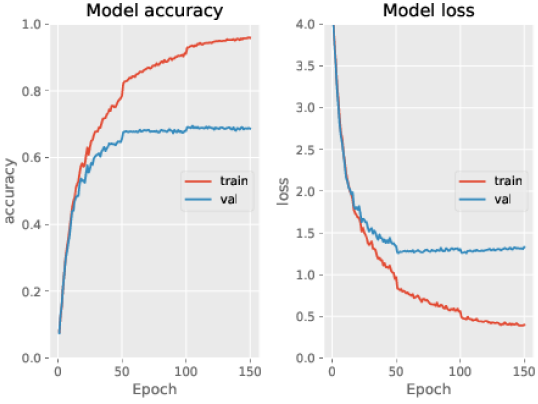
with Augmentation

- with input augmentation: HFlip, Scale, Trans,
- Observe that **overfitting was reduced**.

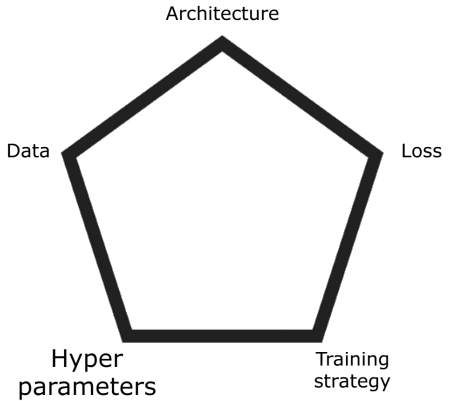


with Regularization

- L2 (0.0025), Dropout(0.5), Label smoothing(0.1)
- Observe that **overfitting was reduced**.



Model Debugging



Each pillar—architecture, loss, data, hyperparameters, training strategy—can critically impact performance.

Architecture Design

- **Depth vs Width:** Find the right trade-off between model complexity and performance - we already discussed about increasing depth and width (number fo filters/neurons per layer).
- **Task-specific modules:** Many architectures are built for specific goals:
  - Autoencoders: compression, denoising, generation
  - GANs: high-fidelity image generation
  - UNet: segmentation
- Multi-branch networks help extract diverse features and handle data more flexibly.

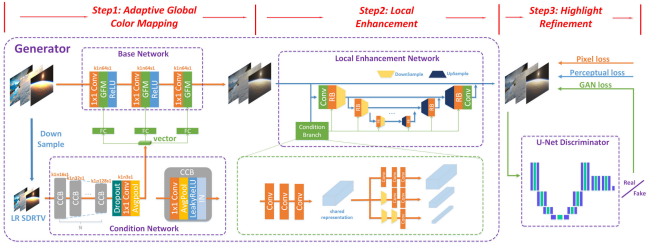
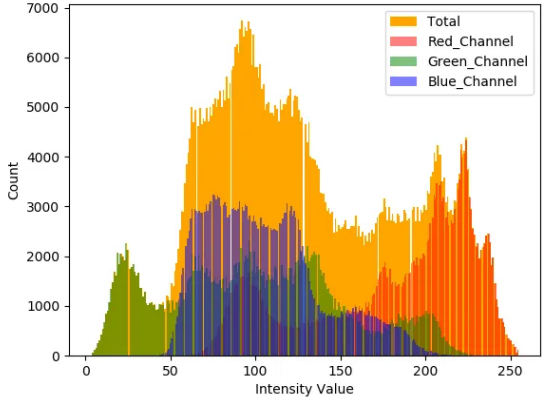


Figure: Visualization from Chen et al., *Towards Efficient SDRTV-to-HDRTV by Learning from Image Formation*, arXiv:2309.04084 (2023).

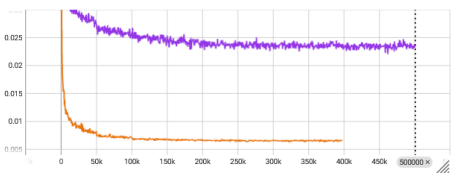
Data Quality and Distribution

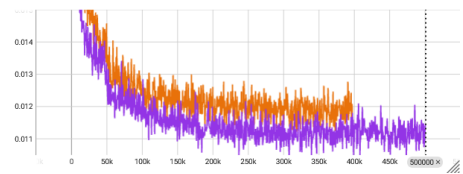
- Data quality is often overlooked — garbage in, garbage out.
- Datasets are split into: train, validation, and test sets.
- **Why both validation and test?** Validation is used during training, test is held out for final evaluation.
- Ideally: all datasets are IID (independent and identically distributed) — often not the case!
- **Histogram analysis** helps uncover distribution mismatches.
- Generalization vs Specialization: open this up for discussion/examples.



Loss Function Design

- The loss function defines the task: classification, segmentation, generation, etc.
- In real projects, the loss is usually a combination of multiple terms.
- **In theory:** each loss term supervises a different aspect of learning.
- **In practice:** one loss can dominate — masking model flaws.
- Plot individual loss components and tune weights accordingly.



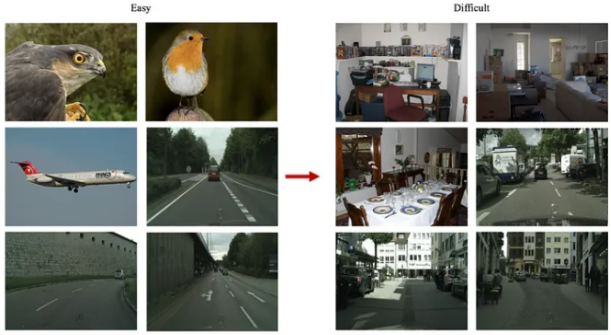


Hyperparameters and Optimization

- **Key hyperparameters:**
  - Learning Rate: most sensitive; use schedulers like cosine annealing, OneCycle.
  - Batch Size: smaller batches = better generalization, more noise.
  - Weight Decay: important for regularizing optimizers like AdamW.
- **Optimizers:** SGD, Adam, AdamW — each with trade-offs.
- **Tuning tools:** Grid search, Random search, Optuna (Bayesian).

Training Strategies

- **Curriculum Learning:** Start with easier samples, progress to harder ones.
- **Early Stopping:** Stops training when validation performance stops improving.
- **Mixed Precision Training:** Faster training, allows larger batch sizes.
- **Warmup Phase:** Gradually increase learning rate — helps with stability in early training.



Ablation Studies

- Start with a strong baseline model — known to work reliably.
- Add or remove one element at a time: layer, loss, augmentation, etc.
- Run isolated experiments to understand the impact of each change.
- Helps pinpoint what's helping (or hurting) performance.

Component Changed	Performance	Impact
Baseline (full)	92.5%	—
No dropout	88.3%	Overfitting increases
No skip connections	86.7%	Training becomes unstable
No pretraining	81.2%	Poor convergence
Freeze backbone	89.1%	Slight drop, faster training